

# Segment Trees with AtCoder Library

A concise reference for `atcoder::segtree` and `atcoder::lazy_segtree`.

## Part 1 — `atcoder::segtree`

Use when you need **point updates** and **range queries**. If you need range updates too, jump to Part 2.

### What you must define

ACL's `segtree` is templated on three things: the value type `S`, a binary merge `op`, and an identity `e`.

```
using S = long long;

S op(S a, S b) { return a + b; } // merge two children
S e()          { return 0;      } // identity for op

atcoder::segtree<S, op, e> seg(n);
```

### Properties `op` must satisfy (monoid)

- **Associativity**:  $op(op(a,b), c) = op(a, op(b,c))$
- **Left identity**:  $op(e(), x) = x$
- **Right identity**:  $op(x, e()) = x$

Commutativity is *not* required — but if `op` is not commutative, make sure you order arguments correctly: `op(left_child, right_child)`.

### Functions

| Function                           | Complexity  | What it does                                                |
|------------------------------------|-------------|-------------------------------------------------------------|
| <code>set(i, x)</code>             | $O(\log n)$ | Replace value at index <code>i</code> with <code>x</code> . |
| <code>get(i)</code>                | $O(1)$      | Return value at index <code>i</code> .                      |
| <code>prod(l, r)</code>            | $O(\log n)$ | Return $op(a[l], a[l+1], \dots, a[r-1])$ . Half-open.       |
| <code>all_prod()</code>            | $O(1)$      | Return <code>op</code> over the whole array.                |
| <code>max_right&lt;g&gt;(l)</code> | $O(\log n)$ | Largest <code>r</code> such that $g(prod(l, r))$ is true.   |
| <code>min_left&lt;g&gt;(r)</code>  | $O(\log n)$ | Smallest <code>l</code> such that $g(prod(l, r))$ is true.  |

**max\_right** is a binary-search-on-segtree primitive: it walks the tree comparing running products against the predicate `g`, costing only one  $\log$  factor instead of  $\log^2 n$ . The predicate must be *monotone*: if `g` is true for some prefix it must be true for every smaller prefix.

### Example — Range Sum, Point Update

```
#include <atcoder/segtree>
using namespace atcoder;

using S = long long;
S op(S a, S b) { return a + b; }
S e()          { return 0; }

segtree<S, op, e> seg(n);
seg.set(3, 7);           // a[3] = 7
long long s = seg.prod(2, 9); // sum of a[2..8]
long long all = seg.all_prod(); // sum of full array
```

### Variant — Range Max

```
S op(S a, S b) { return max(a, b); }
S e()          { return LLONG_MIN; } // identity for max
```

## Part 2 — atcoder::lazy\_segtree

Use when you need **range updates** combined with **range queries**. Range updates are deferred as *lazy tags* sitting on internal nodes and pushed down only when needed.

### What you must define

Two types and five functions. S is the value, F is the tag type.

```
using S = ...;    // node value type
using F = ...;    // lazy tag type

S op(S a, S b);           // merge children
S e();                   // identity for op
S mapping(F f, S x);      // apply tag f to value x
F composition(F f, F g);  // f after g (f is newer)
F id();                  // do-nothing tag

atcoder::lazy_segtree<S, op, e, F, mapping, composition, id> seg(n);
```

### The 5 laws

All five must hold or the tree will silently produce wrong answers.

#### 1. op is a monoid

```
op(op(a,b), c) == op(a, op(b,c))
op(e(), x)      == x
op(x, e())      == x
```

#### 2. id is the identity tag

```
mapping(id(), x) == x
```

#### 3. composition has id as identity

```
composition(id(), f) == f
composition(f, id()) == f
```

#### 4. composition is consistent with mapping

```
mapping(composition(f,g), x) == mapping(f, mapping(g, x))
```

Read this as: applying g then f gives the same result as applying their composition once. This is what lets pending tags collapse safely.

#### 5. mapping distributes over op

```
mapping(f, op(a,b)) == op(mapping(f,a), mapping(f,b))
```

Essential for **range queries**. A tag sitting on an internal node must have the same effect on its stored aggregate as it would after being pushed down to every leaf. Easy to forget — for range-add + range-sum this is exactly why a node's tag application must multiply by segment length.

### Argument order (the part everyone gets wrong)

In both `mapping(f, x)` and `composition(f, g)`, the **first argument is the newer operation**. Think of it as f acting on x, or f layered on top of g.

Concrete example for tag = (mul, add) meaning  $x \rightarrow x \cdot \text{mul} + \text{add}$ :

```
// g applied first: x -> x*g.mul + g.add
// then f applied: -> (x*g.mul + g.add)*f.mul + f.add
//                  = x*(f.mul*g.mul) + (f.mul*g.add + f.add)

F composition(F f, F g) {
    return { f.mul * g.mul,
             f.mul * g.add + f.add };
}
```

## Functions

| Function                           | Complexity  | What it does                                                                 |
|------------------------------------|-------------|------------------------------------------------------------------------------|
| <code>set(i, x)</code>             | $O(\log n)$ | Replace <code>a[i]</code> with <code>x</code> . Pushes tags down on descent. |
| <code>get(i)</code>                | $O(\log n)$ | Return <code>a[i]</code> . Pushes tags down on descent.                      |
| <code>prod(l, r)</code>            | $O(\log n)$ | Return <code>op(a[l..r-1])</code> . Pushes on the path.                      |
| <code>all_prod()</code>            | $O(1)$      | Return <code>op</code> over the whole array.                                 |
| <code>apply(i, f)</code>           | $O(\log n)$ | Apply tag <code>f</code> to single position <code>i</code> .                 |
| <code>apply(l, r, f)</code>        | $O(\log n)$ | Apply tag <code>f</code> to range <code>[l, r)</code> . The deferred update. |
| <code>max_right&lt;g&gt;(l)</code> | $O(\log n)$ | Binary search; pushes tags along the path.                                   |
| <code>min_left&lt;g&gt;(r)</code>  | $O(\log n)$ | Symmetric to <code>max_right</code> .                                        |

## How each call mutates the tree

### apply(l, r, f) — deferred update

- Walk down from root toward [l, r). At each **ancestor of the range**, push pending tag to children before descending.
- At nodes **fully inside [l, r)**, do *not* descend. Instead: `node.val = mapping(f, node.val);`  
`node.tag = composition(f, node.tag);`
- On the way back up, recompute each ancestor: `node.val = op(left.val, right.val)`.

Only  $O(\log n)$  nodes are touched. Most of the range's leaves are never visited — their updates live as tags on a handful of internal nodes.

### get(i), prod(l, r) — queries

- Walk from root to the relevant nodes. At every step, **push the current node's tag to its children** before descending into either child.
- After pushing, the current node's tag becomes `id()`.
- The leaf or covered nodes are now up-to-date; the result is read (or merged) cleanly.

### Push-down step in detail

When a node has tag `f` and children `L`, `R`:

```
L.val = mapping(f, L.val);  
L.tag = composition(f, L.tag);
```

```
R.val = mapping(f, R.val);  
R.tag = composition(f, R.tag);
```

```
node.tag = id();
```

The child either had no pending tag — in which case its tag becomes `f` directly — or it had its own pending `g`, which now becomes `composition(f, g)`. This is the exact reason `composition` exists.

### set(i, x) — point assign

- Push tags down on the path from root to leaf `i`.
- Overwrite the leaf.
- Recompute every ancestor on the way back up with `op`.

### Tag merging — visual

Suppose tag `g = +2` already sits on a node and tag `f = +5` arrives:

|                 |    |                                 |
|-----------------|----|---------------------------------|
| before          |    | after apply(f)                  |
| [val=10, tag=2] | -> | [val=15, tag=7]                 |
| /     \         |    | /     \                         |
| ...    ...      |    | ...    ... (children untouched) |

```
val: mapping(f, val) = 10 + 5 = 15  
tag: composition(f, tag) = 5 + 2 = 7
```

Children stay stale — they still encode "add 2 not yet applied". When the tree is eventually pushed down, children receive `composition(f, g) = +7`, which is exactly the total they were owed.

## Example A — Range Add, Range Sum

Classic lazy use case. Value type is a (sum, length) pair so that a range-add of  $v$  contributes  $v \cdot \text{length}$  to the sum.

```
#include <atcoder/lazysegtree>
using namespace atcoder;

struct S { long long sum; int len; };
using F = long long;

S op(S a, S b)          { return {a.sum + b.sum, a.len + b.len}; }
S e()                  { return {0, 0}; }
S mapping(F f, S x)     { return {x.sum + f * x.len, x.len}; }
F composition(F f, F g) { return f + g; }
F id()                 { return 0; }

int n; cin >> n;
vector<S> init(n, {0, 1});          // each leaf has length 1
lazy_segtree<S, op, e, F, mapping, composition, id> seg(init);

seg.apply(2, 7, 3);                  // a[2..6] += 3
long long s = seg.prod(0, n).sum;    // sum of full array
```

Notice how the **distributivity law** dictates the form of mapping: applying tag  $f$  to a node covering  $\text{len}$  leaves must shift the sum by  $f \cdot \text{len}$ , not just by  $f$ .

## Example B — Range Assign, Range Max

Range assignment needs a **sentinel** for `id()` because zero (or any real value) is a legitimate assignment target. Common choice: `LLONG_MIN`.

```
#include <atcoder/lazysegtree>
using namespace atcoder;

using S = long long;
using F = long long;

const F NONE = LLONG_MIN;          // sentinel: no pending assign

S op(S a, S b)          { return max(a, b); }
S e()                  { return LLONG_MIN; }
S mapping(F f, S x)     { return f == NONE ? x : f; }
F composition(F f, F g) { return f == NONE ? g : f; } // newer wins
F id()                 { return NONE; }

lazy_segtree<S, op, e, F, mapping, composition, id> seg(n);
seg.apply(l, r, v);          // a[l..r-1] = v
long long m = seg.prod(l, r); // max over a[l..r-1]
```

Key idea: assignment *clobbers* any older tag, so composition simply returns the newer one (unless it's the sentinel).

## Combined updates — range assign + range add

The tag is now a pair: *first set, then add*. State: (has\_assign, assign\_val, add\_val). Mapping applies assign before add. Composition with a newer assign discards the older add. Skip the code here — the pattern follows directly from the 5 laws.

## Final notes

- ACL is half-open:  $[l, r)$ . Always.

- Vectors passed to the constructor become the initial leaves. The default constructor fills with `e()`.
- If queries return wrong answers, suspect law 5 first (distributivity) — it's the most commonly violated.
- Non-commutative `op` is fine; just make sure mapping treats the two children consistently and you call `op(left, right)` in that order.
- Beware of `id()` == `real_value` collisions. Use a sentinel or pack a flag.